Class Derivation

Advanced Programming

Brent van Bladel

Classes

A **class** is user-defined type providing an abstraction to a concept.

- Group data that forms a larger concept together.
- Link functionality to data.

```
1.  class Reporter {
2.    public:
3.        Reporter();
4.        ~Reporter() = default;
5.
6.        void print(std::string_view msg);
7.  };
```

Classes

A **class** is user-defined type providing an abstraction to a concept.

- Group data that forms a larger concept together.
- Link functionality to data.

```
1.  class Reporter {
2.    public:
3.        Reporter();
4.        ~Reporter() = default;
5.
6.        void print(std::string_view msg);
7.  };
```

A concept does not exist in isolation.

Relationships between classes

Association

- Aggregation (“X has a Y” relationship)
- Composition (“X is part of Y” relationship)

Generalization (“X is a Y” relationship)

Aggregation ("X has a Y" relationship)

- Related object can exist independently of each other.
- Weak association.
- Implemented by reference or shared pointer.

```
1.  class Reporter {
2.    public:
3.        Reporter();
4.        ~Reporter() = default;
5.
6.        void print(std::string_view msg);
7.
8.     private:
9.         std::ostream& _out;
10. };
```

Composition ("X is part of Y" relationship)

- Related object cannot exist independently of each other.
- Strong association.
- Implemented by data member or unique pointer.

```
1.  class Circle{
2.    public:
3.        Circle(Coordinate c);
4.        ~Circle() = default;
5.     private:
6.         std::unique_ptr<Coordinate> _center;
7.  };
```

Generalization ("X is a Y" relationship)

Implemented by subclassing.

Terminology

- Superclass | Base class
- Subclass | Derived class

Goals

- Interface inheritance
- Implementation reuse

```
1.  class Circle : public Shape{
2.    public:
3.        Circle(Coordinate c);
4.        ~Circle() = default;
5.    private:
6.        std::unique_ptr<Coordinate> _center;
7.  };
```

Interface Inheritance

Different derived classes can be used through the interface of a common base class.

```
1.  class Shape {
2.   public:
3.     virtual float area() const { return 0; };
4.     virtual float perimeter() const { return 0; };
5.  };
6.
7.  class Circle : public Shape {
8.   public:
9.     float area() const override { return 3.14 * _radius * _radius; };
10.    float perimeter() const override { return 2 * 3.14 * _radius; };
11.  private:
12.        float _radius{1};
13. };
14.
15. class Square: public Shape {
16.  public:
17.    float area() const override { return _length * _length ; };
18.    float perimeter() const override { return 4 * _length ; };
19.  private:
20.        float _length{1};
21. };
```

Implementation Reuse

Derived classes can reuse facilities from the base class.

```
1.  class Rectangle: public Shape {
2.      float _length{1};
3.      float _width{1};
4.   public:
5.     Rectangle(float l, float w) : _length{l}, _width{w} {};
6.     float area() const override { return _length * _width; };
7.     float perimeter() const override { return 2 * (_length  + _width); };
8.  };
9.
10. class Square: public Rectangle{
11.  public:
12.    Square(float l) : Rectangle{l,l} {};
13. };
```

Data Members

- Derived class inherits all data members.
- Derived class can introduce new members.
- Derived class cannot access private members of the base class, only public and protected.

```
1.  class Base {
2.    public:      int x;
3.    protected: int y;
4.    private:     int z;
5.  };
6.
7.  class Derived : public Base {
8.    int a;
9.    // also contains x, y, and z
10.   // can access x and y
11.   // cannot access z
12. };
```

Data Members

Private members of the base class must be initialized via a constructor of the base class.

```
1.  class Base {
2.    public:
3.      Base(int a) : x{a} {};
4.    private:
5.      int x;
6.  };
7.
8.  class Derived : public Base {
9.    public:
10.     Derived(int a) : Base{a} {};
11. };
```

Access Control

Public members can be accessed by anyone.

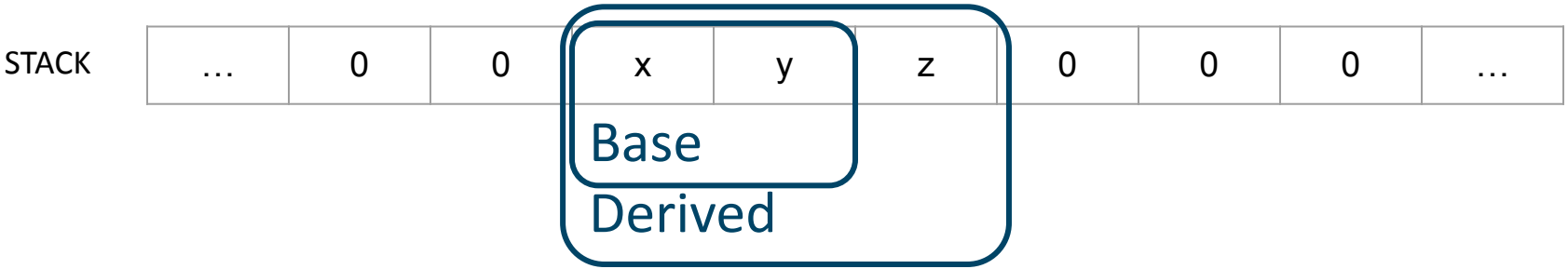
Protected members can be accessed by subclasses.

Private members can be accessed by no one.

```
1.  class A {
2.    public:      int x;
3.    protected: int y;
4.    private:     int z;
5.  };
6.  class B : public A {
7.    // x is public
8.    // y is protected
9.    // z is not accessible
10. };
11. class C : protected A {
12.   // x is protected
13.   // y is protected
14.   // z is not accessible
15. };
16. class D : private A {
17.   // x is private
18.   // y is private
19.   // z is not accessible
20. };
```

Memory Layout of a Derived Class

```
1.  class Base {
2.      int x;
3.      int y;
4.  };
5.
6.  class Derived : public Base {
7.      int z;
8.  };
```



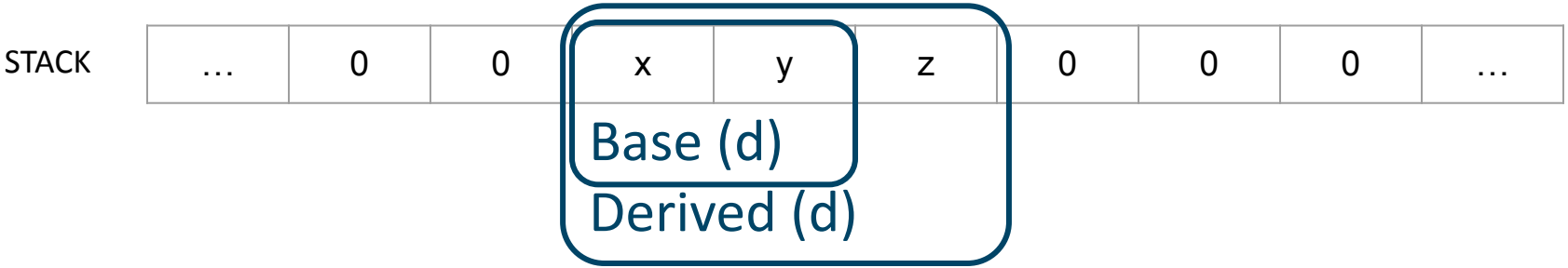
Construction & Destruction

Construction happens from Base to Derived.
Destruction happens from Derived to Base.

```
1.  int main() {
2.      Derived d;
3.  }
```

1. Constructor Base
2. Constructor Derived
3. Destructor Derived
4. Destructor Base

```
1.  class Base {
2.      int x;
3.      int y;
4.  };
5.
6.  class Derived : public Base {
7.      int z;
8.  };
```

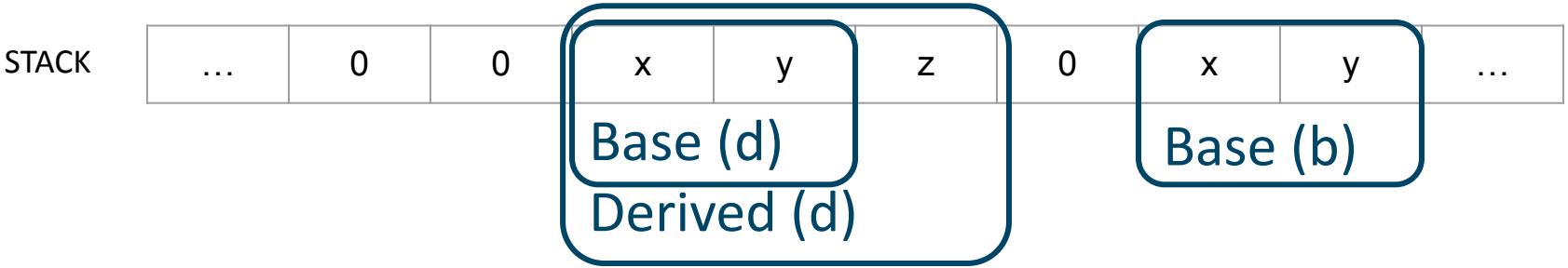


Slicing

```
1.  class Base {
2.      int x;
3.      int y;
4.  };
5.
6.  class Derived : public Base {
7.      int z;
8.  };
```

```
1.  int main() {
2.      Derived d;
3.      Base b = d;
4.  }
```

b copy-constructed from the base of d



Virtual, Override, Final

A function marked **virtual** indicates that a derived class can redefine it.

A function marked **override** indicates that it is virtual and that it redefines the function from the base class.

A function marked **final** indicates that it is virtual, that it redefines the function from the base class, and that it cannot be overridden by a deriving class.

```
1.  class Shape {
2.   public:
3.     virtual float area() const;
4.  };
5.
6.  class Rectangle: public Shape {
7.   public:
8.     float area() override const;
9.  };
10.
11. class Square: public Rectangle{
12.  public:
13.    float area() final const;
14. };
```

Hiding

Hiding

If a derived class redefines function members of the base class, then all the base class function members with the same name become hidden in the derived class.

Member Lookup

1) look in the class scope of this.

2) If not found, go up in the inheritance chain.

3) Stop when the name is encountered.

Final Class

A class marked **final** indicates that it cannot be derived.

Deriving from a final class will result in a compilation error.

```
1.  class Shape {
2.   public:
3.     virtual float area() const;
4.  };
5.
6.  class Rectangle final : public Shape {
7.   public:
8.     float area() override const;
9.  };
10.
11. // compilation error
12. class Square: public Rectangle{
13.  public:
14.     float area() final const;
15. };
```

Pure Virtual

A function marked **pure virtual** indicates that every derived class must redefine it.

A class containing a pure virtual function cannot be instantiated.

A class containing at least one pure virtual function is called an **abstract class**.

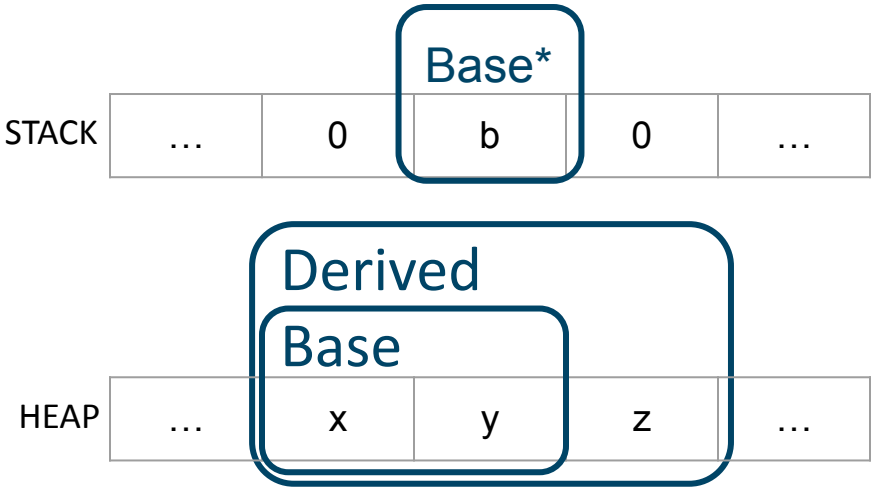
A class containing only pure virtual functions is called an **interface class**.

```
1.  class Shape {
2.   public:
3.    virtual float area() const = 0;
4.  };
5.
6.  class Rectangle: public Shape {
7.   public:
8.     float area() override const;
9.  };
10.
11. class Square: public Rectangle{
12.  public:
13.    float area() final const;
14. };
```

Polymorphism

Since a derived object contains an object of the base class, a pointer to the base class can point to an object of the derived class.

(both objects start on the same address)



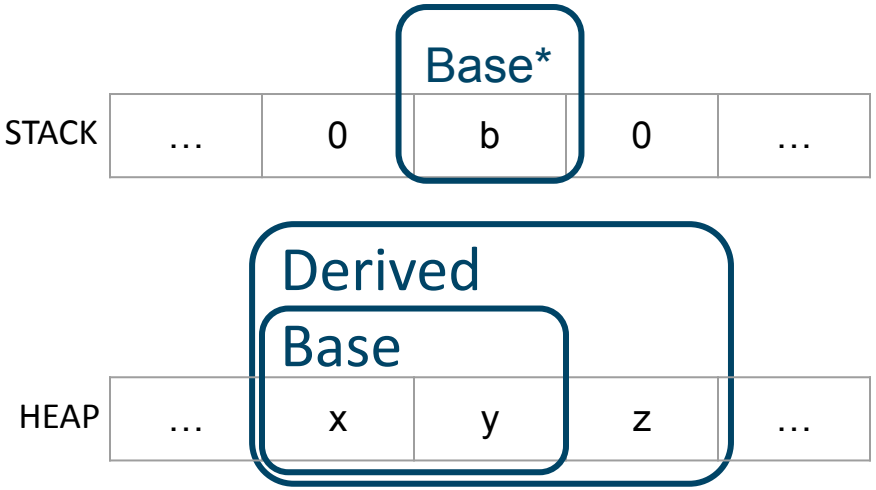

```
1.  class Base {
2.    int x;
3.    int y;
4.   public:
5.    virtual void print() {
6.      std::cout << x << y << std::endl; };
7.  };
8.
9.  class Derived : public Base {
10.   int z;
11.  public:
12.   void print() override {
13.     std::cout << x << y << z << std::endl; };
14. };
15.
16. int main (){
17.   Base* b = new Derived{};
18.   b->print();
19. }
```

Polymorphism

How does the program know at runtime which function to execute?

object of the base class, a pointer to the base class can point to an object of the derived class.

(both objects start on the same address)

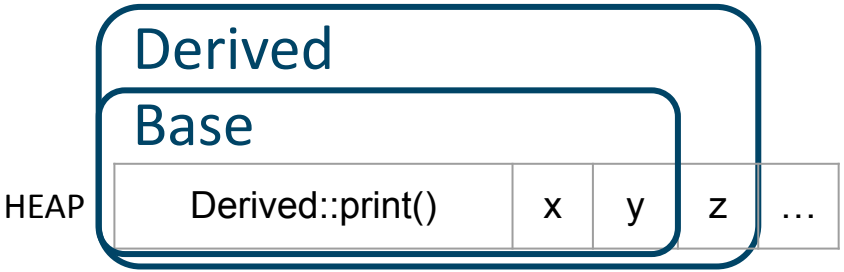



```
1.  class Base {
2.    int x;
3.    int y;
4.   public:
5.    virtual void print() {
6.      std::cout << x << y << std::endl; };
7.  };
8.
9.  class Derived : public Base {
10.   int z;
11.  public:
12.   void print() override {
13.     std::cout << x << y << z << std::endl; };
14. };
15.
16. int main (){
17.   Base* b = new Derived{};
18.   b->print();
19. }
```

vtable

How does the program know at runtime which function to execute?

For each virtual function in the base class, an entry is made in the vtable with the address of the function.
(i.e. the address in static ROM memory where the instructions are stored)

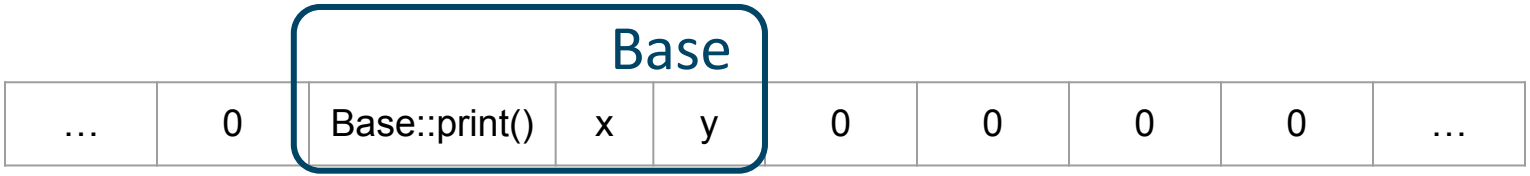



```
1.  class Base {
2.    int x;
3.    int y;
4.   public:
5.    virtual void print() {
6.      std::cout << x << y << std::endl; };
7.  };
8.
9.  class Derived : public Base {
10.   int z;
11.  public:
12.   void print() override {
13.     std::cout << x << y << z << std::endl; };
14. };
15.
16. int main (){
17.   Base* b = new Derived{};
18.   b->print();
19. }
```

vtable

Construction happens from Base to Derived.

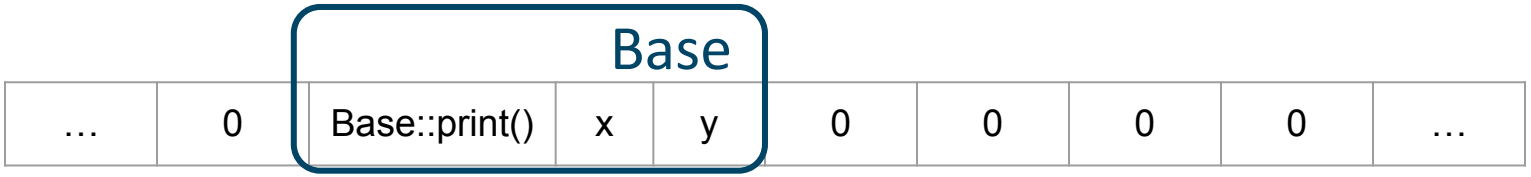
1) Base constructor places the virtual function print in the vtable:



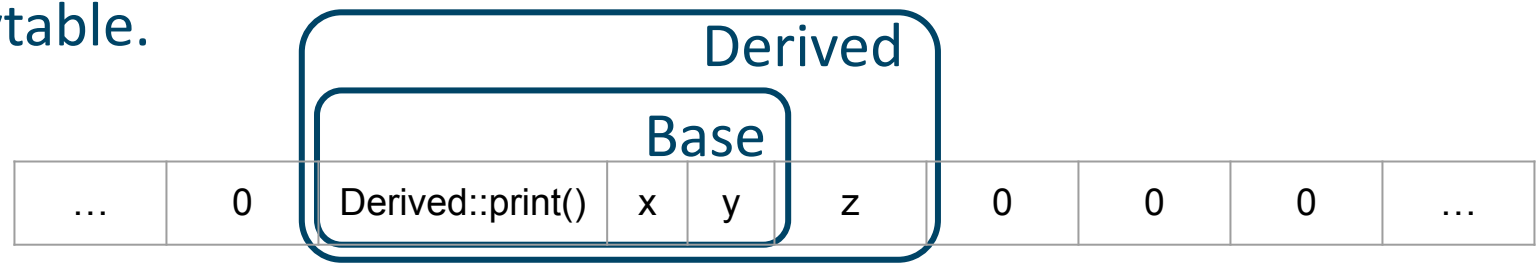
vtable

Construction happens from Base to Derived.

1) Base constructor places the virtual function print in the vtable:



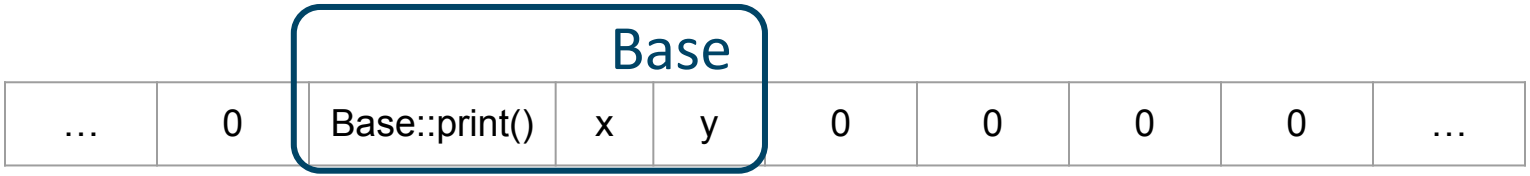
2) Derived constructor overrides the virtual function, and updates the vtable.



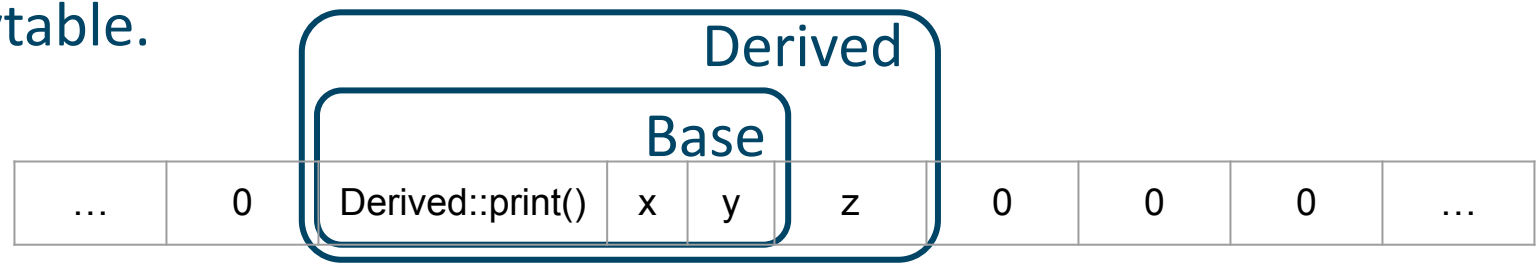
vtable

Construction happens from Base to Derived.

1) Base constructor places the virtual function print in the vtable:



2) Derived constructor overrides the virtual function, and updates the vtable.



3) A function call to a virtual function will check the vtable.

Virtual destructor

A destructor should always be virtual to avoid memory leaks!

```
1.  class Base {
2.   public:
3.     Base() = default;
4.     ~Base() = default;
5.  };
6.
7.
8.  class Derived : public Base {
9.     int* x;
10.  public:
11.    Derived() : x{new int(42)} {};
12.    ~Derived() { delete x; };
13. };
14.
15. int main() {
16.    Base* b = new Derived();
17.    delete b; // memory leak
18. }
```

Inheritance Design

A good object-oriented design follows:

- The Liskov Substitution Principle
- Type Conformity
- The Principle of Closed Behavior

Liskov Substitution Principle

“If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program”

Liskov Substitution Principle

“If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program”

Practically

Code that uses pointers to base classes must be able to use objects of derived classes without knowing it.

Liskov Substitution Principle

```
1.  class Shape{
2.   public:
3.     virtual float area() const = 0;
4.  };
5.
6.  class Line: public Shape{
7.     float _lenght{1};
8.   public:
9.     float area() const override {throw "Line has no area"; };
10. };
11.
12. int main() {
13.     vector<Shape *> = getAllShapes();
14.     double totalArea = 0;
15.     for (Shape* shape : shapes)
16.         totalArea = totalArea  + shape->area();
17. }
```

Line violates LSP!

Type Conformity

“If S is a subtype of T, an object of type S can be provided in any context where an object of type T is expected while preserving correctness.”

Type Conformity

“If S is a subtype of T, an object of type S can be provided in any context where an object of type T is expected while preserving correctness.”

Practically

A derived class should respect the base class’ pre/post-conditions and invariants.

Type Conformity

```
1.  class Shape{
2.   public:
3.     //postcondition: returns area > 0
4.     virtual float area() const = 0;
5.  };
6.
7.  class Line: public Shape{
8.     float _lenght{1};
9.   public:
10.    float area() const override {return 0 };
11. };
12.
```

Line is not type conform with Shape!

Principle of Closed Behavior

“If S is a subtype of T, the behavior inherited by S from T should preserve correctness in S.”

Principle of Closed Behavior

“If S is a subtype of T, the behavior inherited by S from T should preserve correctness in S.”

Practically

Inherited functionality should respect the derived class’ pre/post-conditions and invariants.

Principle of Closed Behavior

```
1.  class Rectangle : public Shape {
2.    float _width;
3.    float _length;
4.   public:
5.    Rectangle(float w, float l) : _width{w}, _length{l} {};
6.    void setWidth(float w) {_width = w; };
7.    void setLength(float l){_length = l; };
8.  };
9.
10. class Square: public Rectangle {
11.  public:
12.    //invariant: length == width
13.    Square(float l) : Rectangle{l,l} {};
14. };
```

Principle of closed behavior violation!